

would block replenishment." That mechanism does not exist here. `READ COMMITTED` is still mandatory, for an unrelated reason measured in §1.3 — but a design document that carries the MySQL rationale across would be teaching the wrong model.

2 — There are no secondary index locks to avoid. A PostgreSQL row lock lives in the heap tuple header, not in an index. "Locking strictly the required physical index tuples" describes a problem the engine does not have. The composite key still matters enormously — as an *access path*, and because every extra index multiplies write amplification on a table doing 60,000 inserts

and deletes a minute.

3 — SQL comment tags do not aggregate in `pg_stat_statements`. Three queries carrying two different `/* conn_tag:... */` values collapsed into one row with `calls = 3`, comment stripped. So did `application_name`. The proposed observability approach silently measures nothing; §5 has what works instead.

§1 Schema, and the index that is an access path

1.1 The three tables

The article names the pool key `(shop_id, inventory_item_id, inventory_group_id, id)`; the DDL below uses `location_id` in that third position, matching how the question was posed. Substitute freely — the shape is what matters.

► `inventory_ledger` — authoritative balance

```
CREATE TABLE inventory_ledger (
    shop_id          bigint      NOT NULL,
    inventory_item_id bigint      NOT NULL,
    location_id       bigint      NOT NULL,
    on_hand           integer     NOT NULL,
    committed         integer     NOT NULL DEFAULT 0,
    -- Tokens minted into the pool and not yet returned. A minted token is
    -- either free in reservation_units or held in reserved_quantities.
    pool_issued       integer     NOT NULL DEFAULT 0,
    updated_at        timestampz NOT NULL DEFAULT now(),

    PRIMARY KEY (shop_id, inventory_item_id, location_id),

    CONSTRAINT ledger_non_negative
        CHECK (on_hand >= 0 AND committed >= 0 AND pool_issued >= 0),

    -- The oversell guard. Enforced by the database, not by the application
    -- remembering to check: minting and settlement both pass through it.
    CONSTRAINT ledger_pool_within_stock
        CHECK (committed + pool_issued <= on_hand)
) WITH (fillfactor = 70);
```

`fillfactor = 70` belongs *here* and nowhere else in this schema. This row is updated on every mint and every settlement; leaving free space on the page is what lets those

be HOT updates that never touch the index. §4 shows why the same setting on the token table is cargo cult.

► reservation_units – the bounded token pool

```
CREATE TABLE reservation_units (
    shop_id          bigint NOT NULL,
    inventory_item_id bigint NOT NULL,
    location_id       bigint NOT NULL,
    unit_id           bigint GENERATED ALWAYS AS IDENTITY,

    PRIMARY KEY (shop_id, inventory_item_id, location_id, unit_id)
) WITH (
    -- No fillfactor: this table is INSERT and DELETE only, never UPDATE, so
    -- there is no HOT path to leave room for. Reserving free space would add
    -- pages to scan and buy nothing. Measured in §4: n_tup_hot_upd = 0.
    autovacuum_vacuum_scale_factor = 0.0,
    autovacuum_vacuum_threshold    = 200,
    autovacuum_vacuum_cost_delay   = 0,
    autovacuum_vacuum_cost_limit   = 2000,
    autovacuum_analyze_scale_factor = 0.0,
    autovacuum_analyze_threshold   = 2000
);
```

One index. That is the design. The primary key is the only access path the claim query needs, and adding a second index would double the write amplification on the highest-churn table in the system. The three leading columns are equality predicates; `unit_id` exists to make the key unique, not to be searched.

MEASURED — 402,000 TOKENS ACROSS 402 SKUS

```
EXPLAIN (ANALYZE, BUFFERS, COSTS OFF)
SELECT ctid FROM reservation_units
WHERE shop_id=1 AND inventory_item_id=1234 AND location_id=10
LIMIT 3 FOR UPDATE SKIP LOCKED;

Limit (actual time=0.120..0.124 rows=3.00 loops=1)
  Buffers: shared hit=10
  -> LockRows (actual time=0.118..0.122 rows=3.00 loops=1)
    -> Index Scan using reservation_units_pkey on reservation_units
        Index Cond: ((shop_id = 1) AND (inventory_item_id = 1234)
            AND (location_id = 10))
        Index Searches: 1
        Buffers: shared hit=7
  Execution Time: 0.817 ms
```

One index search, ten buffers, no sort. With a single SKU loaded the planner correctly

One index search, ten buffers, no sort. With a single SKD loaded the planner correctly chooses a sequential scan instead — a plan captured on a toy dataset would have proved nothing.

► reserved_quantities – active holds

```
CREATE TABLE reserved_quantities (
    reservation_id    uuid          PRIMARY KEY DEFAULT uuidv7(),    -- PG18 built-in
    checkout_id       uuid          NOT NULL,
    shop_id           bigint        NOT NULL,
    inventory_item_id bigint        NOT NULL,
    location_id       bigint        NOT NULL,
    quantity          integer       NOT NULL CHECK (quantity > 0),
    state             text          NOT NULL DEFAULT 'held'
                                CHECK (state IN ('held','settled','released')),
    expires_at        timestampz    NOT NULL,
    created_at        timestampz    NOT NULL DEFAULT now(),

    -- Idempotency. A retried checkout call must not double-hold, and network
    -- retries on a checkout path are not an edge case.
    CONSTRAINT reserved_once_per_line
        UNIQUE (checkout_id, inventory_item_id, location_id)
) WITH (fillfactor = 80);

-- The expiry sweeper's only access path. Partial, so settled and released
-- rows leave the index entirely instead of sitting in it forever.
CREATE INDEX reserved_expiring_idx
    ON reserved_quantities (expires_at) WHERE state = 'held';
```

`uuidv7()` is new in PostgreSQL 18 and worth taking: a v4 key scatters every insert across the whole index, while v7's millisecond prefix keeps inserts appending to one side of it. Same opacity in a URL, materially cheaper to maintain.

1.2 One row per unit, or one row per line?

The article says one row per sellable unit throughout. The DDL above keeps that for the *pool* and aggregates in the *hold*: one `reserved_quantities` row per (checkout, item, location) carrying `quantity`, rather than five rows for a cart line of five.

Tokens are fungible — nothing distinguishes token #41 from token #42, and release re-mints rather than returning specific identities. So per-unit hold rows cost five times the WAL and five times the index churn to preserve information the system never reads. The invariant survives either way, and stays checkable:

```
-- pool_issued must equal free tokens plus held tokens, for every key.
-- Anything else means a token was minted and lost, or consumed twice.
CREATE VIEW pool_invariant AS
SELECT l.shop_id, l.inventory_item_id, l.location_id,
       l.on_hand, l.committed, l.pool_issued,
       COALESCE(u.free, 0) AS free_tokens,
       COALESCE(h.held, 0) AS held_tokens,
       l.pool_issued = COALESCE(u.free,0) + COALESCE(h.held,0) AS balanced
FROM inventory_ledger l
LEFT JOIN (SELECT shop_id, inventory_item_id, location_id, count(*) AS free
           FROM reservation_units GROUP BY 1,2,3) u USING (shop_id, inventory_i
LEFT JOIN (SELECT shop_id, inventory_item_id, location_id, sum(quantity)::bigint
           FROM reserved_quantities WHERE state='held' GROUP BY 1,2,3) h USING
```

1.3 Why **READ COMMITTED** — the PostgreSQL reason

The conclusion ports. The reason does not.

MySQL needs it to escape gap locking. PostgreSQL needs it because a **REPEATABLE READ** transaction that meets a row changed since its snapshot **cannot lock it and must abort**. Under **READ COMMITTED**, each statement takes a fresh snapshot and **FOR UPDATE** re-evaluates the row against the newest committed version — the mechanism PostgreSQL calls EvalPlanQual. Under **REPEATABLE READ** there is nothing to re-evaluate against; the transaction is holding a snapshot it cannot abandon.

MEASURED — SESSION A CLAIMS FROM A POOL SESSION B IS DRAINING

```
READ COMMITTED      snapshot saw 100, B took 50, A claimed 10
REPEATABLE READ     snapshot saw 100, B took 50, A FAILED
                    SQLSTATE=40001  could not serialize access due to
                    concurrent update
```

During a flash sale the pool is being drained continuously by definition, so under **REPEATABLE READ** this is not a rare race — it is the steady state. Every checkout would 40001 and retry.

The behaviour as the pool approaches zero is the benign case: **SKIP LOCKED** never waits, so a sweep that finds every remaining token locked returns fewer rows than asked for, immediately. Partial fulfilment is a normal return value, not an error, and the caller decides — the reference implementation rolls back the line and reports the shortfall rather than selling three of five.

§2 The reserve path and the single-writer refuel

2.1 Claim and hold, in fixed statement order

Every path in the system touches these tables in the same order —

`reservation_units` then `reserved_quantities`, and settlement walks `reserved_quantities` then `inventory_ledger`. No cycle can form because no path runs the sequence backwards.

```
CREATE OR REPLACE FUNCTION reserve_line(
    p_checkout uuid, p_shop bigint, p_item bigint, p_loc bigint,
    p_want integer, p_ttl interval DEFAULT '15 minutes'
) RETURNS integer LANGUAGE plpgsql AS $$
DECLARE v_got integer;
BEGIN
    WITH claimed AS (
        SELECT ctid
        FROM reservation_units
        WHERE shop_id = p_shop
        AND inventory_item_id = p_item
        AND location_id = p_loc
        -- No ORDER BY. Tokens are fungible, so any p_want will do, and
        -- omitting it lets the index scan stop the moment it has enough.
        LIMIT p_want
        FOR UPDATE SKIP LOCKED
    ),
    taken AS (
        DELETE FROM reservation_units u
        USING claimed c
        WHERE u.ctid = c.ctid
        RETURNING 1
    )
    SELECT count(*)::integer INTO v_got FROM taken;

    IF v_got = 0 THEN RETURN 0; END IF;

    INSERT INTO reserved_quantities
        (checkout_id, shop_id, inventory_item_id, location_id, quantity, expires_at
    VALUES (p_checkout, p_shop, p_item, p_loc, v_got, now() + p_ttl)
    ON CONFLICT (checkout_id, inventory_item_id, location_id) DO UPDATE
        SET quantity = reserved_quantities.quantity + EXCLUDED.quantity,
            expires_at = EXCLUDED.expires_at;

    RETURN v_got;
END $$;
```

WHY `ctid` IS SAFE HERE, AND WHERE IT WOULD NOT BE

Joining on `ctid` instead of the four-column key is much cheaper, and it is safe for one specific reason: **this table is never updated**. A tuple's `ctid` changes when the row is updated and a new version is written elsewhere; with only INSERT and DELETE, a locked tuple's identity is stable for the life of the transaction. On any table that takes UPDATES, join on the primary key instead.

MEASURED — FLASH SALE, 40 CONCURRENT SHOPPERS, 50 UNITS IN STOCK

```
ledger on_hand=50   minted=50   pool=50
40 shoppers x 3 units = 120 demanded

outcome: {'sold': 48, 'carts': 16, 'short': 24}
ledger : on_hand=50 committed=48 pool_issued=2
pool   : free=2   invariant_balanced=True

RESULT: no oversell, invariant holds
```

Note the pool capped itself at 50 despite a target of 1,000: the ledger's headroom is what bounds minting, so the cap is a consequence of the CHECK constraint rather than a number the application has to respect.

2.2 Replenishment: wait, don't scatter

The article is specific that concurrent reserves *"wait for it to finish rather than all racing to insert rows."* That distinction matters more than it looks. A non-blocking `pg_try_advisory_xact_lock` makes the losers give up — and a shopper told "sold out" while stock exists is a worse failure than a shopper who waited 8 ms.

```
CREATE OR REPLACE FUNCTION replenish_pool(
    p_shop bigint, p_item bigint, p_loc bigint, p_target integer DEFAULT 1000
) RETURNS integer LANGUAGE plpgsql AS $$
DECLARE v_key bigint; v_have integer; v_headroom integer; v_mint integer;
BEGIN
    v_key := hashtextextended(p_shop::text||':'||p_item::text||':'||p_loc::text, 0)

    -- Blocking, and *_xact_* without exception. See §5 for why the session
    -- form is a correctness bug under a transaction-mode pooler.
    PERFORM pg_advisory_xact_lock(v_key);
```

```

-- Re-check after acquiring: the transaction that held the lock probably
-- already refuelled, and this is the branch 23 of 24 callers take.
SELECT count(*) INTO v_have FROM reservation_units
  WHERE shop_id=p_shop AND inventory_item_id=p_item AND location_id=p_loc;
IF v_have >= p_target THEN RETURN 0; END IF;

-- The ledger row lock costs nothing here: the advisory lock guarantees
-- there is no queue behind it.
SELECT on_hand - committed - pool_issued INTO v_headroom
  FROM inventory_ledger
  WHERE shop_id=p_shop AND inventory_item_id=p_item AND location_id=p_loc
  FOR UPDATE;
IF NOT FOUND OR v_headroom <= 0 THEN RETURN 0; END IF;

v_mint := LEAST(p_target - v_have, v_headroom);

UPDATE inventory_ledger
  SET pool_issued = pool_issued + v_mint, updated_at = now()
  WHERE shop_id=p_shop AND inventory_item_id=p_item AND location_id=p_loc;

INSERT INTO reservation_units (shop_id, inventory_item_id, location_id)
SELECT p_shop, p_item, p_loc FROM generate_series(1, v_mint);

RETURN v_mint;
END $$;

```

MEASURED — 24 CONCURRENT REFUELS RELEASED FROM A BARRIER

```

blocking  pg_advisory_xact_lock      -> {'minted': 1, 'found_full': 23}
non-block pg_try_advisory_xact_lock -> {'minted': 1, 'backed_off': 23}

both:  pool = 1000  pool_issued = 1000  (no double-mint)

```

Both prevent the stampede and neither double-mints. They differ in what the other 23 shoppers experience: *found_full* means they go on to claim tokens, *backed_off* means they were told there were none. Use the blocking form on the checkout path; the try-form is for background top-up jobs, where giving up is free.

THE ONE HAZARD IN THIS FUNCTION

`pg_advisory_xact_lock` blocks with no timeout. A refuel that stalls — a slow `generate_series` against a huge target, a hung ledger row — queues every checkout for that SKU behind it. Set `SET LOCAL lock_timeout` before the call, or bound the mint size, and treat lock-wait as a first-class metric rather than discovering it during a sale. PostgreSQL 18's `log_lock_failures` logs the failures when you do set a timeout.

§3 Multi-SKU carts in one round trip

The question asks how to batch a cart without cross-item deadlocks. The honest first answer is that `SKIP LOCKED` cannot deadlock on the token table at all — deadlock requires two transactions waiting on each other, and this one never waits. Ordering discipline is needed only where the system genuinely blocks: the ledger row during replenishment, and the unique index on `reserved_quantities`. Order by `(inventory_item_id, location_id)` there and the cycle cannot form.

For the round trip itself, `CROSS JOIN LATERAL` over an ordered array does the whole cart in one statement. Whether PostgreSQL accepts `FOR UPDATE` in that position is the kind of thing worth checking rather than assuming:

```
WITH requested AS (
    SELECT * FROM unnest($2::bigint[], $3::bigint[], $4::integer[])
        AS t(inventory_item_id, location_id, want)
),
ordered AS (
    -- Deterministic across every cart in the fleet.
    SELECT * FROM requested ORDER BY inventory_item_id, location_id
),
claimed AS (
    SELECT o.inventory_item_id, o.location_id, u.ctid
    FROM ordered o
    CROSS JOIN LATERAL (
        SELECT ru.ctid
        FROM reservation_units ru
        WHERE ru.shop_id = $1
            AND ru.inventory_item_id = o.inventory_item_id
            AND ru.location_id = o.location_id
        LIMIT o.want
        FOR UPDATE SKIP LOCKED
    ) u
),
taken AS (
    DELETE FROM reservation_units ru USING claimed c
    WHERE ru.ctid = c.ctid
    RETURNING ru.inventory_item_id, ru.location_id
)
SELECT inventory_item_id, location_id, count(*) AS claimed
FROM taken GROUP BY 1,2 ORDER BY 1,2;
```

MEASURED — CART OF THREE SKUS REQUESTED OUT OF ORDER (300, 100, 200)

inventory_item_id	location_id	claimed
100	10	3
200	10	1
300	10	2

Accepted, and claimed exactly the requested 3 / 1 / 2 in one round trip, in key order regardless of the order the cart supplied.

The caller still has to compare `claimed` against `want` per line and decide the cart's fate. An all-or-nothing cart rolls back the whole transaction, which returns every token at once — the cheapest possible compensation, and another reason the tokens are fungible.

§4 Churn: what actually bloats, and what does not

4.1 `fillfactor` on the token table is cargo cult

`fillfactor` exists to leave room on a page so an UPDATE can write the new tuple version beside the old one and skip the index update — a HOT update. A table that only ever takes INSERT and DELETE performs no HOT updates, so there is nothing for the free space to enable. Lowering it just adds pages to scan.

MEASURED — 60,000 INSERTS AND DELETES ON `RESERVATION_UNITS`

```
n_tup_ins = 61,210   n_tup_del = 60,060   n_tup_hot_upd = 0
```

Zero, as the table definition guarantees. Put the `fillfactor` on `inventory_ledger`, where updates actually happen, and leave the pool at the default.

4.2 The heap comes back; the index reaches a ceiling

This is the finding that justifies the 1,000 cap on grounds the article does not state.

MEASURED — CHURN, THEN VACUUM, THEN 60,000 MORE OPERATIONS

POINT IN THE RUN	HEAP	INDEX	DEAD TUPLES
After 60k ins/del, before vacuum	3,520 KB	2,440 KB	60,000
After <code>VACUUM</code>	16 KB	2,440 KB	0
+20,000 more, vacuumed	16 KB	3,264 KB	0
+40,000 more, vacuumed	16 KB	3,264 KB	0
+60,000 more, vacuumed	16 KB	3,264 KB	0

The heap is fully reclaimed — 3,520 KB down to 16 KB — because the dead pages are at the end of the relation and vacuum truncates them. The index is not: B-tree pages are recycled for reuse, never returned to the operating system. So the index settles at its **high-water mark and stops**. 180,000 further operations moved it not one byte.

That ceiling is the real argument for bounding the pool. Index size here is a function of the peak number of live tokens, not of throughput. Cap the pool at 1,000 per item/location and the index for that SKU is permanently bounded no matter how many millions of units flow through it. Remove the cap and there is no ceiling to reach.

4.3 The binding constraint is naptime, not thresholds

MEASURED — AUTOVACUUM DURING THE BURST

Across 60,000 inserts and deletes with `autovacuum_vacuum_scale_factor = 0.0` and `threshold = 200`, autovacuum ran **zero times**. Not because the thresholds were wrong — they were met thousands of times over — but because the burst finished inside one `autovacuum_naptime`, which defaults to 60 seconds.

Tuning the thresholds on a burst-y table and leaving naptime alone tunes the wrong knob. A flash sale lasting ninety seconds gets at most one autovacuum pass.

SETTING	VALUE	WHY
<code>autovacuum_naptime</code>	10s	Cluster-wide. The single highest-leverage change for burst workloads; the default of 60 s is the reason a tuned table still bloats.

<code>autovacuum_vacuum_scale_factor</code>	0.0	Per-table. A percentage of a small table is a small number of tuples — use a flat threshold instead of a proportion.
<code>autovacuum_vacuum_threshold</code>	200	Per-table. Vacuum this table often and cheaply rather than rarely and expensively.
<code>autovacuum_vacuum_cost_delay</code>	0	Per-table. Never throttle the worker on the pool; it is small and hot, and a throttled pass is worse than none.
<code>autovacuum_vacuum_cost_limit</code>	2000	Per-table. Let it finish in one pass.
<code>autovacuum_vacuum_max_threshold</code>	default	New in PG 18: an absolute <i>cap</i> on the scale-factor calculation, defaulting to 100,000,000. It earns its keep on large tables where 20 % of rows is an absurd number — not on a bounded pool, where scale factor is already zeroed.
<code>autovacuum_max_workers</code>	raise	Cluster-wide. Bounded pools across thousands of SKUs mean many small tables' worth of work; three workers becomes the queue.

On replication lag: aggressive vacuuming of a small table generates little WAL relative to the reservation traffic itself, and the pool is bounded, so a pass is cheap. The lag risk in this architecture is not vacuum — it is the `generate_series` insert during replenishment, which writes 1,000 tuples in one transaction. Batch the mint if replicas are tight, and note that `hot_standby_feedback = on` will hold back vacuum on the primary, which on this table is precisely the wrong trade.

§5 Connection hold time, and measuring it honestly

The article's own conclusion is that the database was never the constraint: cleaning up the checkout path *"removed 50% of reads and 33% of transactions on the primary,"* after which writer CPU stayed under 50 % and reader CPU under 16 %. The bottleneck was how long each checkout held a connection. That finding transfers completely — and PostgreSQL, with its process-per-connection model, is if anything less forgiving.

5.1 Pooling mode is a correctness question, not a tuning one

CONSTRUCT	TRANSACTION MODE	WHY
<code>FOR UPDATE SKIP LOCKED</code>	SAFE	Row locks are held by the transaction and released at COMMIT, which is exactly when the pooler hands the connection on.
<code>pg_advisory_xact_lock</code>	SAFE	Released with the transaction, by definition.
<code>pg_advisory_lock</code> (session)	UNSAFE	Outlives the transaction. The pooler gives that backend to another tenant, who inherits a lock they never took and cannot release. This is the single most common way a token-pool design corrupts itself.
<code>SET</code> (session-level)	UNSAFE	Leaks to the next borrower. Use <code>SET LOCAL</code> inside the transaction — this includes <code>lock_timeout</code> and <code>application_name</code> .
Session-scope temp tables, cursors	UNSAFE	Same leak, same reason.

Run PgBouncer in **transaction mode**, and treat the session-scoped constructs above as a lint rule rather than a convention — a session advisory lock in a transaction-pooled system fails silently and intermittently, which is the worst way for anything to fail. Keep `default_pool_size` well under the backend's `max_connections`: the point of the pooler is that PostgreSQL never sees the concurrency, and a pool sized to match the fleet defeats it.

The architectural rule the article arrives at is worth restating plainly: **open the transaction as late as possible and close it as early as possible**. Payment authorisation, fraud scoring, tax calculation and address validation must all happen outside it. The reservation transaction should contain the claim, the hold, and nothing else.

5.2 The tagging approach in the brief does not work

MEASURED — THREE QUERIES, TWO DIFFERENT `CONN_TAG` VALUES

```
/* conn_tag:checkout_reserve */ SELECT count(*) FROM t WHERE sku = 1;
/* conn_tag:admin_report */      SELECT count(*) FROM t WHERE sku = 2;
/* conn_tag:checkout_reserve */ SELECT count(*) FROM t WHERE sku = 3;
```

queryid	calls	query
-3337652484301807207	3	SELECT count(*) FROM t WHERE sku = \$1

One row. Three calls. The comment is gone from the stored text entirely. Setting `application_name` instead produced the same collapse — one row, two calls.

The reason is structural: `pg_stat_statements` aggregates on `(userid, dbid, toplevel, queryid)`, and `queryid` is computed from the post-parse tree, which contains no comments. Nothing you put in a comment can ever split a row there. A dashboard built on this premise reports a single blended latency for every caller of a shared query and looks entirely plausible while doing it.

WHAT TO USE INSTEAD

The comment is not useless — it just lands somewhere else. It survives verbatim in `pg_stat_activity.query` and in the server log, which is where per-transaction attribution actually belongs:

MEASURED — SAMPLING A TAGGED QUERY MID-FLIGHT

application_name	query	held
checkout:reserve	SET application_name='checkout:reserve'; /* conn_tag:checkout_reserve */ SELECT ...	00:00:01.239762

- **Hold time, per business transaction.** Sample `pg_stat_activity` on a timer and histogram `now() - xact_start` grouped by `application_name`. This measures the thing the article found to be the bottleneck, which `pg_stat_statements` structurally cannot: total time attributable to *the transaction*, not to individual statements.
- **Tag with `SET LOCAL application_name`** at the top of each transaction — transaction-scoped, so it cannot leak through the pooler. Put `%a` in `log_line_prefix` and every log line carries it

`log_min_duration_statement` and every log line carries it.

- **Keep the SQL comment as well**, for the tools that read raw text: PgBouncer's `log`, `auto_explain`, and `log_min_duration_statement`.
- **Use `pg_stat_statements` for what it is good at** — which statement shapes cost the most in aggregate — and stop asking it which caller ran them.
- **Watch the pooler, not just the database.** PgBouncer's `SHOW POOLS` reports `cl_waiting` and `maxwait`. Clients queuing for a connection is the symptom the article describes, and it does not appear in any database-side CPU metric.

§6 When this is the wrong design

The token pool solves one specific problem: many concurrent writers contending for one counter row. It is worth naming what it costs, because the architecture is frequently adopted by systems that do not have that problem.

SITUATION	VERDICT
Flash sales, hundreds of concurrent checkouts per SKU	USE IT — this is the case it was built for, and nothing simpler survives the contention.
Steady traffic, single-digit concurrency per SKU	DON'T — a single row with <code>UPDATE ... WHERE available >= n</code> is correct, one statement, and has no replenishment path to get wrong.
Long holds (quotes valid for days)	DON'T — tokens are a short-lived cache of the ledger. Holding thousands for days makes the pool the system of record by accident.
Inventory adjusted frequently from outside checkout	CAREFUL — every downward adjustment must reconcile against minted tokens, and the ledger CHECK will reject the write instead of silently overselling. That is the right failure, but the operator-facing path has to handle it.

Two operational commitments come with adopting it. The **expiry sweeper is not optional**: tokens held by abandoned carts are stock that cannot be sold, and without a sweeper a flash sale ends with a pool full of holds for checkouts nobody completed. And the **invariant check has to run** — `pool_invariant` should be a monitored query, not a debugging view, because a token minted and lost is inventory that silently ceases to exist